김수환 포트폴리오

프로젝트 화면과 구조도를 중심으로 구현 방식과 설계 기준을 설명합니다.

Bitria 암호화폐 브로커리지 플랫폼

2025.11 – 현재 · 운영 중　　웹·iOS·Android·데스크톱 프론트엔드 전담(앱 스토어 출시)·백엔드 API 개발 · 2인 팀

외부 거래소 유동성으로 현물·선물·마진 거래와 계정·자산 관리를 제공하는 암호화폐 브로커리지 플랫폼입니다.

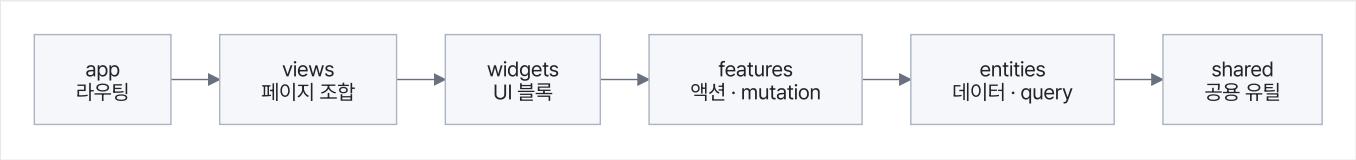
2인 팀에서 프론트엔드 전체를 초기 구축부터 개발하고, 백엔드 API 개발에도 참여했습니다.
아래 자료는 화면·도메인 분리, 실시간 데이터 전달, 멀티 플랫폼 구성을 설명합니다.

구현 구조와 설계

FSD 아키텍처와 화면 분기 기준

화면과 도메인이 늘어나도 의존성이 뒤섞이지 않도록 app → views → widgets → features → entities → shared 순의 단방향 규칙을 정했습니다.
조회(query)는 entity에, 변경(mutation)은 feature에 두어 각 레이어의 책임을 분리했습니다.

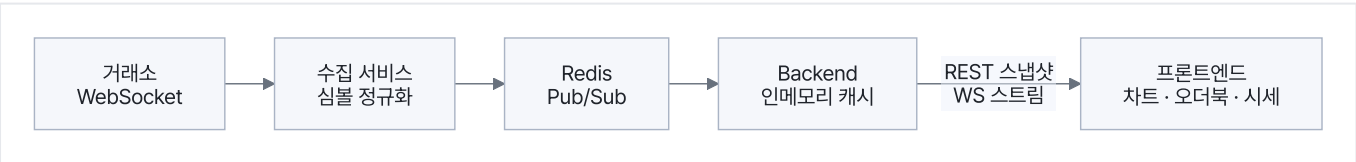
웹과 앱의 라우트가 다르면 폴더를 분리했습니다.
화면 크기 차이가 작으면 prop으로 분기하고, 마크업 차이가 크면 디바이스별 파일을 두도록 기준을 정했습니다.



FSD 레이어 간 의존 방향과 각 레이어의 책임

실시간 데이터 전달 구조

거래소 WebSocket 데이터는 수집 서비스의 심볼 정규화와 Redis Pub/Sub을 거쳐 백엔드에 전달됩니다.
프론트엔드는 REST 스냅샷과 WebSocket 스트림으로 차트·호가·시세를 표시합니다.

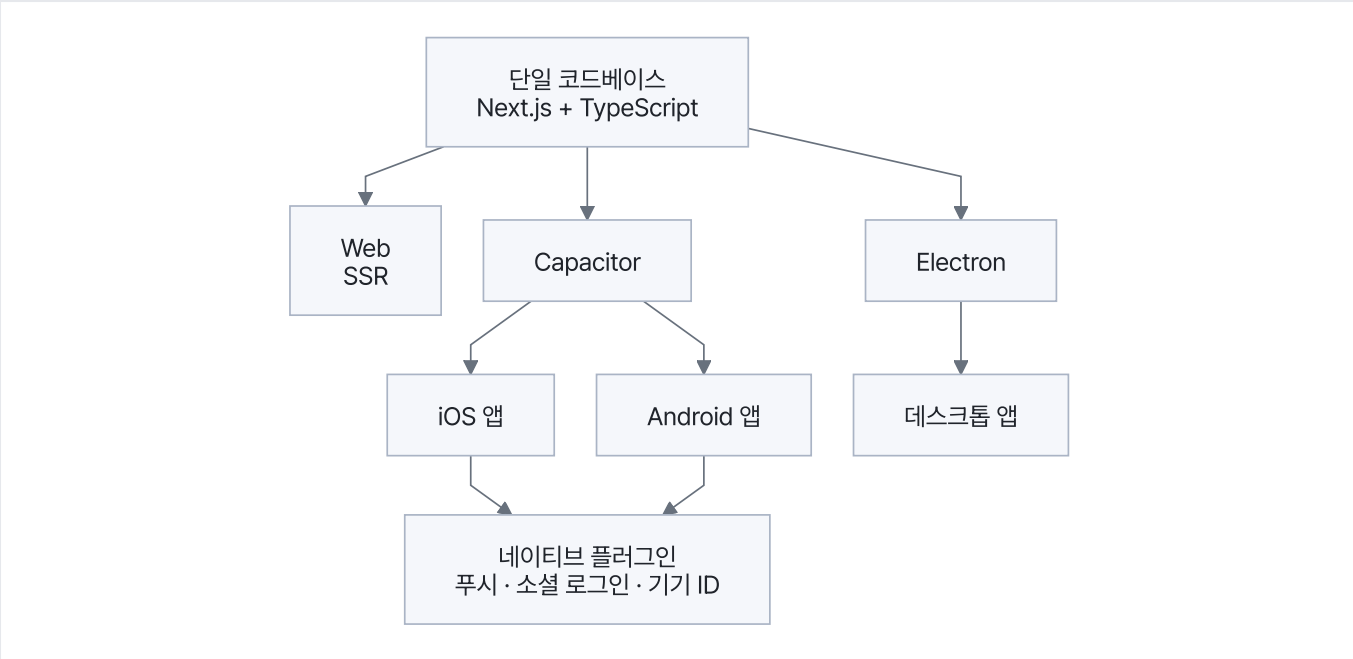


실시간 데이터 전달 경로

단일 코드베이스의 멀티 플랫폼 대응

2인 팀에서 웹·iOS·Android·데스크톱을 빠르게 함께 내기 위해, 각 플랫폼을 따로 만들지 않고 기존 Next.js 코드베이스를 그대로 쓰는 Capacitor와 Electron을 선택했습니다.
Next.js·TypeScript 코드베이스를 웹, Capacitor 기반 iOS·Android 앱, Electron 데스크톱 앱으로 빌드했습니다.
모바일 앱을 스토어에 출시하고 푸시(APNs·FCM)·소셜 로그인·기기 식별 기능을 연결했습니다.

앱 강제 업데이트 정책을 연동하고, Electron 앱의 API 요청을 식별하는 헤더를 추가했습니다.



웹·모바일·데스크톱 빌드와 네이티브 기능 연결 구조

문제 해결과 개선

배럴 import의 불필요한 모듈 유입 개선

공통 컴포넌트를 한 경로로 가져오도록 만든 `shared/ui` 배럴이 three.js 컴포넌트까지 다시 내보내, 버튼 하나를 import해도 163개 모듈이 하나의 청크로 묶였습니다.
모바일에서 가장 오래 실행되던 이 청크를 없애려고 배럴 import 136곳을 컴포넌트 정의 파일 직접 참조로 변경했습니다.
아래는 실제 변경한 import의 예시입니다.

변경 전 · 공통 배럴 참조

```
import { SortButton } from '@/shared/ui';
```

변경 후 · 정의 파일 직접 참조

```
import { SortButton } from '@/shared/ui/navigation/sort-icon';
```

symbol-list-header.tsx 변경 예시

> 직접 import 변경 전후의 로컬 프로덕션 빌드를 각 5회 측정한 결과, 모바일 Lighthouse TBT 중앙값은 417ms에서 292.5ms로 감소했습니다.

측정 조건은 로컬 프로덕션 빌드, 모바일 CPU 4배 감속, 네트워크 제한입니다.
three.js 동적 import는 전후 모두 적용돼 있어 직접 import 변경만 비교했습니다.
외부 API 응답과 측정 당시 시스템 부하에 따라 수치는 달라질 수 있습니다.

빌드·번들 최적화

SVG를 컴포넌트와 이미지 주소(`?url`)로 나눠 쓰는데 당시 Turbopack 규칙은 쿼리 분기를 지원하지 않아, 개발 서버는 Turbopack, 프로덕션 빌드는 webpack으로 나눠 운영했습니다.
Next.js 16.2에서 쿼리 조건이 추가된 것을 확인하고 16.4로 올려 빌드를 Turbopack으로 통일했습니다.
불러오는 순간 동작해야 하는 모듈 4개와 CSS만 `sideEffects` 에 명시해 나머지 모듈은 쓰지 않으면 번들에서 빠지도록 했고, 한국어 화면이 없는데도 한글 전체를 받던 Pretendard 폰트는 화면에 필요한 글자 구간만 받는 서브셋으로 바꿨습니다.

부수효과가 있는 모듈만 명시

```
"sideEffects": [
  "**/*.css",
  "./src/entities/symbol/lib/market-socket.ts",
  "./src/shared/lib/notify-socket.ts",
  "./src/shared/api/client.ts",
  "./src/shared/lib/error-report.ts"
]
```

package.json의 sideEffects 선언

> 프로덕션 빌드 시간은 74.2초에서 32.8초로 56%, 홈 JS는 752KB에서 550KB로 27%, 초기 폰트 전송은 2,048KB에서 111KB로 95% 줄었습니다.

빌드 시간은 캐시 없는 프로덕션 빌드를 전후 커밋 교대로 3회 측정한 중앙값입니다.
JS 크기는 프로덕션 빌드 결과물의 gzip 기준, 폰트 전송량은 홈 화면 기준입니다.

보이지 않는 화면이 렌더 시간을 차지하던 문제 해결

앱 거래 화면은 차트를 바로 보여주려고 주문 화면과 차트 화면을 함께 마운트해 두고 보이는 쪽만 전환했습니다.
차트 화면으로 이동한 뒤에야 캔들 조회와 소켓 연결을 시작해 빈 화면을 기다리던 문제를 없애기 위한 설계였습니다.
그 대가로 숨겨진 화면도 시세 갱신마다 다시 렌더링됐고, React가 렌더할 때마다 알려주는 내부 연결 지점(DevTools 훅)에 측정 코드를 붙여 확인해 보니, 보이지 않는 UI가 렌더 시간의 64%(1,028ms 중 653ms)를 차지했습니다.
숨겨진 화면에 구독 비활성화 옵션을 넘겨도 시세 핸들러가 전역 목록으로 호출돼 효과가 없어, 실시간 값을 표시하는 컴포넌트가 직접 구독하도록 분리했습니다.

변경 전 · 화면 최상위에서 시세 구독

```
const [snapshot, setSnapshot] = useState<Snapshot | null>(null);
useSnapshotSocket({
  symbols: symbolInfo ? [symbolInfo] : [],
  onUpdate: handleSnapshotUpdate,
});
```

변경 후 · 표시 컴포넌트가 직접 구독

```
// 실시간 값은 LiveSymbolStats 안에서 구독한다
<LiveSymbolStats
  fallbackPrice={symbolInfo?.snapshot?.price}
  fallbackChange={symbolInfo?.snapshot?.change24h}
/>
```

spot-chart-view.tsx 변경 요약 (실제 코드에서 발췌)

> 동일한 시세 메시지를 12초간 재생한 실험에서 렌더 작업 합계 중앙값이 5,241회에서 1,534회로 71% 줄었습니다.
> 차트를 미리 준비하는 동작은 그대로 유지하면서, 주문 폼·하단 패널·차트 헤더는 시세 갱신 중 다시 렌더링되지 않습니다.

원인 진단 수치(64%)는 개발 빌드에서 실시간 갱신만 12초간 측정한 1회 결과입니다.
개선 결과는 개발 빌드, 모바일 화면(390×844), 같은 공개 시세 메시지 73개 재생, 개선 전후 각 5회 중앙값이며, 기기 부하 차이로 렌더 시간 대신 렌더 횟수를 대표 지표로 사용했습니다.

차트 로딩 실패의 원인 분석과 해결

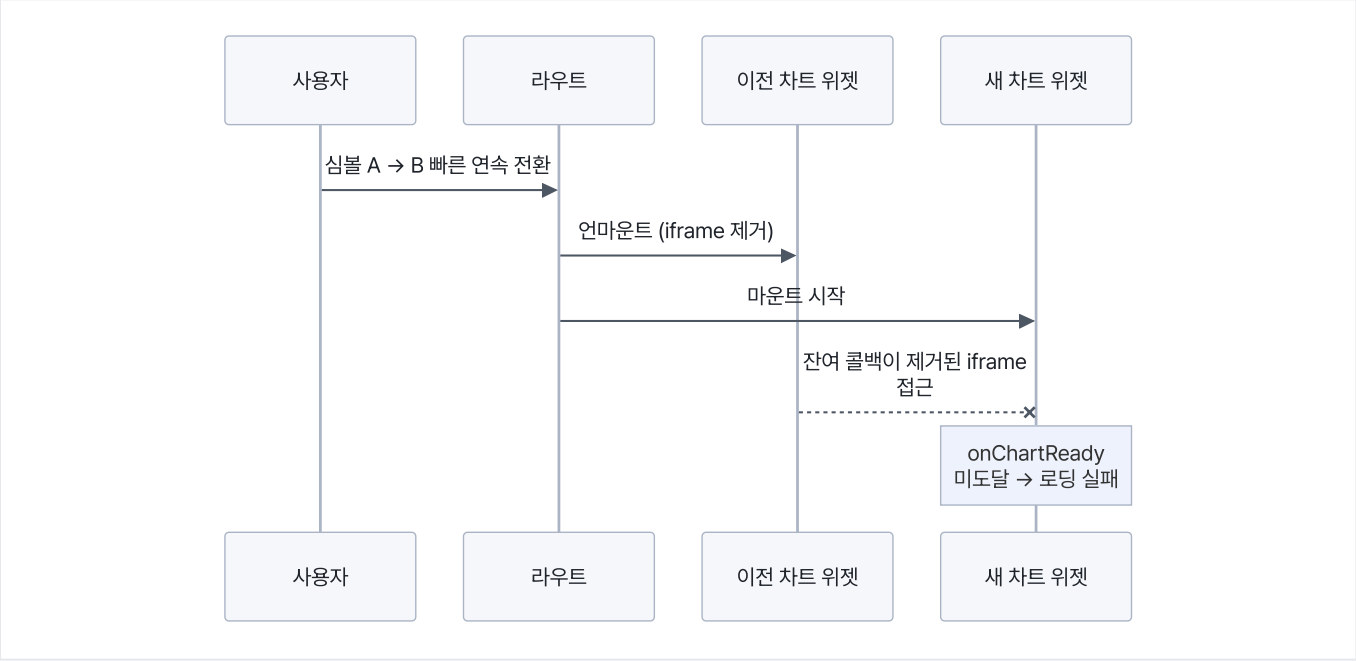
처음에는 차트가 실패하면 로딩 화면에 그대로 멈췄고, 서버에 실패 기록이 한 건도 남지 않았습니다.
TradingView는 실패 신호를 주지 않고, 데이터 요청에는 타임아웃이 없었기 때문입니다.
스크립트 로드 재시도, 응답 없이 멈춘 요청의 타임아웃, 준비 완료 신호 안전망을 추가하고 실패 단계와 원인을 서버 로그로 보고하도록 했습니다.
TradingView가 캔들 요청 실패 뒤 다시 요청하지 않아, 직접 재시도하고 최종 실패 시 재시도 버튼을 보여주도록 했습니다.

수집된 로그에서 13일간 50건 대부분이 앱 거래 화면에서 발생한 것을 확인했습니다.
인앱 브라우저 대응으로 켜 둔 호환 로딩 모드와, Cloudflare 엣지 캐시를 살리려고 1년 캐시로 바꾼 번들이 겹치면서 TradingView 릴리스 노트의 알려진 버그(`onChartReady` 누락) 조건이 만들어졌습니다.
라이선스 제약으로 버전을 올릴 수 없어, blob 주소를 쓰지 않는 `same_origin` iframe 로딩 방식으로 전환했습니다.

빠르게 심볼을 전환할 때 차트 로딩이 완료되지 않는 문제를 클라이언트 오류 로그와 서버·CDN 응답 시간으로 추적했습니다.
이전 위젯에 남아 있던 콜백이 이미 제거된 iframe에 접근하는 상황을 재현했습니다.

심볼 전환마다 차트 위젯이 재마운트되면서 발생하는 생명주기 충돌을 줄이기 위해, 위젯을 거래 레이아웃에서 유지했습니다.
위젯이 준비되기 전에는 최신 심볼을 보관하고, 준비 상태에 따라 반영하도록 변경했습니다.

실패했을 때의 대응도 바꿨습니다.
TradingView는 로드 성공 콜백(`onChartReady`)만 제공해 실패는 25초 타임아웃으로 추정하고 있었습니다.
차트 iframe에 리소스 오류 리스너를 연결해, 스크립트 로드가 실패하면 타임아웃을 기다리지 않고 바로 실패 화면과 재시도 버튼을 보여주도록 했습니다.



수정 전 심볼 전환 경쟁 상태: 이전 위젯에 남아 있던 콜백이 제거된 iframe에 접근해 새 위젯이 준비 완료에 도달하지 못하는 흐름

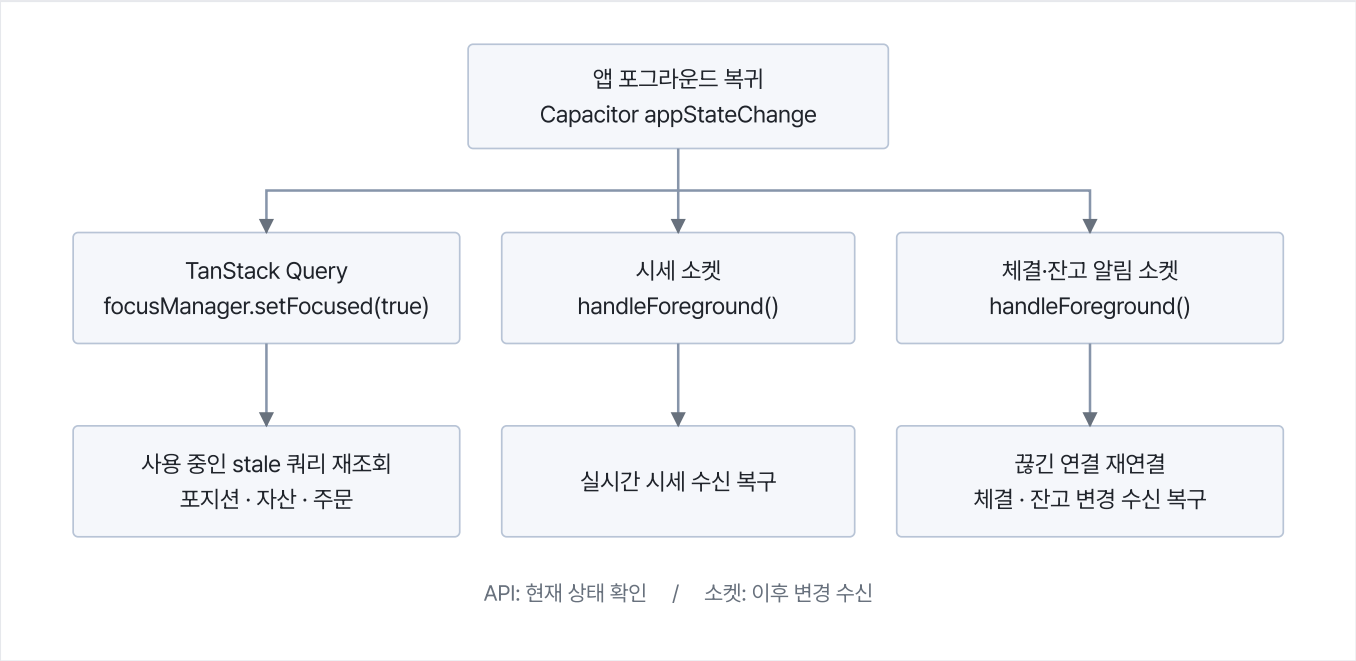
> 빠르게 심볼을 전환해도 마지막에 선택한 심볼로 차트가 정상 로딩되고, 실패하더라도 로딩 화면에 갇히지 않고 재시도로 복구됩니다.
> 실패는 단계별 원인과 함께 운영 로그에 남고, 정상 로드에서는 실패로 오판하지 않는 것을 함께 확인했습니다.

앱 복귀 후 포지션·자산 정보 불일치 해결

앱 복귀 후 시세는 움직이지만 이미 종료된 포지션이 화면에 그대로 남는 문제가 있었습니다.
앞서 복귀 문제를 시세 소켓 기준으로 고쳐, 앱 복귀 이벤트로 복구되는 것은 시세 소켓뿐이었고 API 조회 캐시와 체결·잔고 알림 소켓은 복구되지 않았습니다.

네이티브 `appStateChange`를 공통 진입점으로 사용했습니다. `focusManager.setFocused(isActive)`로 화면에서 사용 중인 stale 쿼리를 다시 조회하고, 시세·알림 소켓의 복귀 처리를 함께 실행합니다.

API 재조회는 복귀 시점의 현재 상태를 확인하고, 소켓 재연결은 이후 변경을 계속 받는 역할입니다.
알림 소켓의 재연결 5회 제한을 제거하고, 이미 연결돼 있거나 로그인한 사용자가 없으면 추가 연결을 생략하도록 했습니다.



앱 복귀 이벤트에서 현재 상태 재조회와 실시간 연결 복구를 함께 실행하는 흐름

> 앱 복귀 시 종료된 포지션이 남지 않고, 포지션·자산이 바로 최신 상태로 표시됩니다.
> 페이지 새로고침 없이 갱신이 필요한 쿼리만 다시 요청합니다.

비영어 기기에서 앱 대신 Safari가 열리던 문제 해결

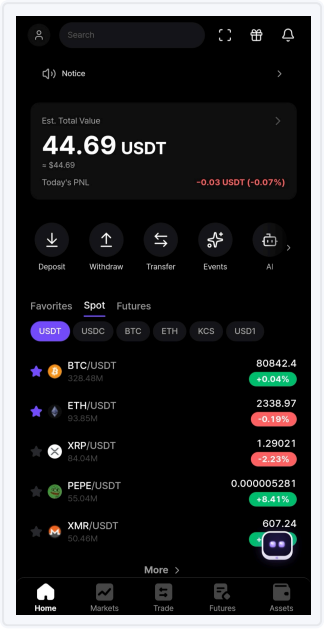
일본어·스페인어·베트남어·태국어·중국어 기기에서 iOS 앱을 실행하면 앱 대신 Safari로 `bitria.com/th/m` 이 열렸습니다.

앱은 Next.js 서버 기능(서버 렌더링·미들웨어)을 그대로 쓰기 위해 원격 URL로 웹을 불러오고, 언어 주소는 기본 언어인 영어만 코드 없이 쓰는 `as-needed` 방식입니다.
Capacitor iOS는 `WebViewDelegationHandler` 에서 이동할 주소가 `server.url` 로 시작하는지 문자열로 비교해 앱 내부 이동을 판정합니다.
`server.url` 은 `https://bitria.com/m` 인데, next-intl의 `localePrefix: 'as-needed'` 설정으로 기본 언어 외에는 `/th/m` 처럼 언어 코드가 앞에 붙어 외부 링크로 판정됐습니다.

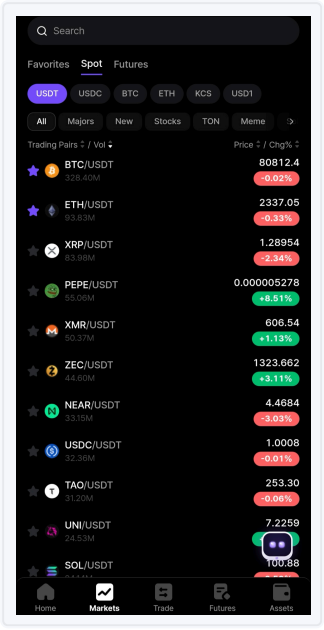
앱 요청(User-Agent의 `BitriaMobile`)에 한해 미들웨어에서 언어 코드를 붙이는 리다이렉트를 같은 대상의 rewrite로 바꿨습니다.
브라우저 주소는 `/m/...` 으로 유지하고 렌더링은 `/{locale}/m/...` 에서 처리해 기기 언어를 그대로 표시합니다.
이미 언어 코드가 붙은 주소로 들어온 앱 요청은 코드 없는 주소로 리다이렉트합니다.

> 영어 외 지원 언어 5개(일본어·스페인어·베트남어·태국어·중국어) 기기에서 앱이 정상 실행됩니다.
> 원격 URL 방식이라 앱 재빌드·스토어 재심사 없이 서버 배포만으로 적용했습니다.

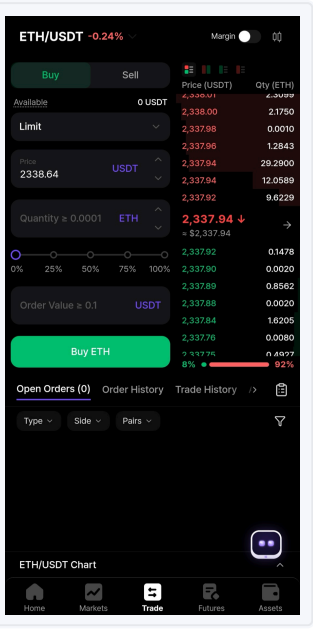
Bitria · 프로젝트 화면



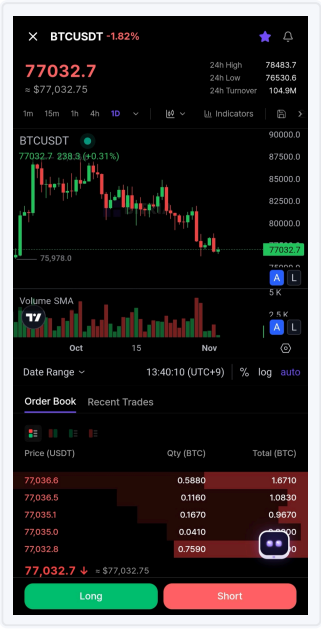
앱 홈 화면



앱 마켓 화면



앱 현물 거래 화면



앱 선물 차트 화면

기술 스택

프론트엔드	TypeScript · Next.js · Tailwind CSS · next-intl · Zustand · TanStack Query · Capacitor · Electron
백엔드	Node.js · Express · Prisma · MariaDB · TimescaleDB · Redis · RabbitMQ · Socket.IO · Docker · Kubernetes(K3s)

Gobit 코인 정보·커뮤니티 플랫폼

2024.10 – 2025.12 프론트엔드·백엔드 개발 담당 · 2인 팀

암호화폐 시세·차트·뉴스·커뮤니티와 모의투자를 제공하는 플랫폼입니다.

2인 팀에서 Next.js 웹 클라이언트와 Node.js·Express API를 개발했습니다.
조회 데이터와 실시간 시세의 전달 경로, 레거시 서비스 통합 구조를 설명합니다.

구현 구조와 설계

실시간 시세와 일반 조회 데이터의 분리

Binance WebSocket으로 수집한 시세를 Redis에 캐시하고 Socket.IO로 웹 클라이언트에 전달했습니다.
시세 이외의 조회 데이터는 SWR로 관리해 실시간 스트림과 API 조회의 역할을 구분했습니다.

TradingView 차트와 tiptap 에디터를 연동하고 회원 인증·게시글·댓글·북마크 기능을 구현했습니다.

레거시 서비스의 통합·배포 구성

다른 개발자가 만든 레거시 서비스는 소스 코드가 유실돼 바이너리로만 서버 5대에 흩어져 있었습니다.
Gobit·모의투자·코인 매매 시그널 서비스를 모노레포로 통합했습니다.
수집기 → Redis → 서버로 데이터 전달 경로를 정리하고, 서비스별 Dockerfile과 Kubernetes 매니페스트를 구성했습니다.

GitHub Actions로 빌드·배포를 자동화하고, 설정값은 Kubernetes Secret에서 불러오도록 구성했습니다.
DB 백업과 캔들 보관 작업은 CronJob으로 분리했습니다.

기술 스택

프론트엔드	TypeScript · Next.js · Tailwind CSS
백엔드	Node.js · Express · MariaDB · Redis · Socket.IO

두잇 홈페이지 리뉴얼 회사 홈페이지 React 마이그레이션

2024.05 – 2024.07 프론트엔드 개발 담당

사내 React 도입에 맞춰 기존 jQuery 기반 회사 홈페이지를 React로 전환하고 새로 단장한 프로젝트입니다.

메인·회사소개·서비스·인증·파트너 페이지를 담당했습니다.
공통 UI 적용과 한국어·영어, 반응형 화면의 구성 방식을 설명합니다.

구현 구조와 설계

공통 UI와 페이지별 구성

색상·타이포·레이아웃 등의 공통 요소에 Doit UI Kit을 적용하고, 페이지별 콘텐츠를 React 컴포넌트로 구성했습니다.

번역 리소스는 i18next의 페이지별 네임스페이스로 분리했습니다.
CSS와 JavaScript에서 같은 브레이크포인트를 사용하고, 모바일에서 불필요한 영역은 렌더링하지 않도록 구성했습니다.

기술 스택

프론트엔드	TypeScript · React · styled-components · i18next

Doit UI Kit 사내 공통 UI 라이브러리

2024.02 – 2024.07 프론트엔드 개발 담당

D.A.X와 회사 홈페이지에 적용한 사내 공통 React UI 라이브러리입니다.

컴포넌트 24종과 패키징·npm 배포·사용 문서를 담당했습니다.
아래 Storybook 화면에서 컴포넌트별 Props와 상태별 UI를 확인할 수 있습니다.

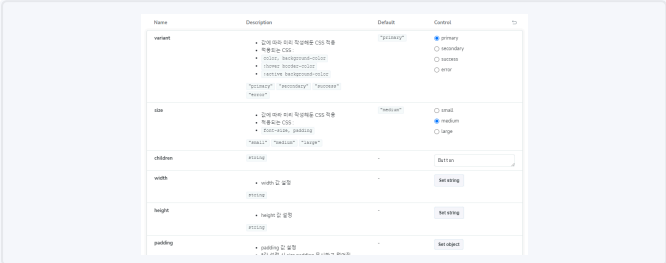
구현 구조와 설계

컴포넌트 조합과 라이브러리 배포

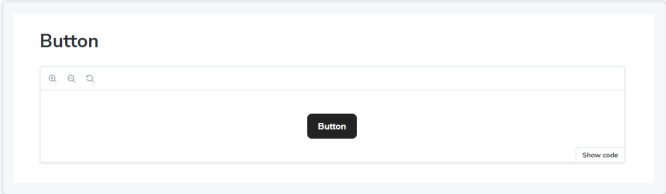
Button·Input 같은 Atom을 조합해 FileInput 등의 Molecule을 구성했습니다.
컴포넌트의 Props와 상태별 UI를 Storybook에 문서화하고, 사용하는 프로젝트에서 같은 기준으로 확인하도록 구성했습니다.
Playground에서는 값을 직접 바꿔 사용 예를 확인할 수 있습니다.

vite-plugin-dts로 타입 선언을 함께 배포하고 React·React DOM은 peer dependency로 분리했습니다.
D.A.X와 회사 홈페이지에 적용하면서 발견한 요구사항을 라이브러리에 반영했습니다.

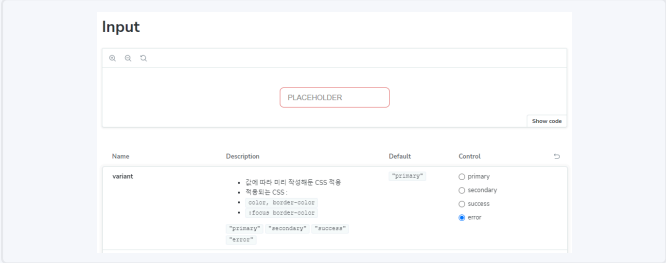
Doit UI Kit · 프로젝트 화면



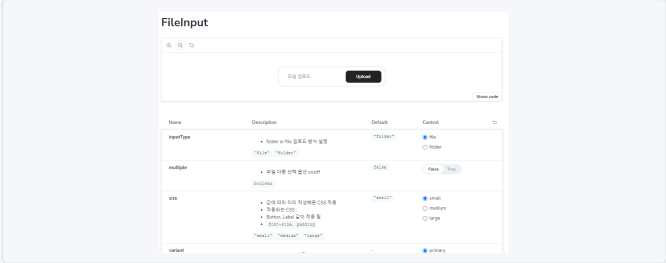
Doit UI Kit 컴포넌트의 Storybook 문서와 Props 설정 화면



Button 컴포넌트의 Storybook 예시



Input 컴포넌트의 상태·Props 문서



Input·Button·Label을 조합한 FileInput 컴포넌트

기술 스택

프론트엔드	TypeScript · React · styled-components · Storybook

D.A.X AI 모델 학습·운영 플랫폼

2024.01 – 2024.06 프론트엔드 개발 담당 · 백엔드 1명과 협업

데이터 관리·라벨링·모델 학습·추론을 지원하는 AI 플랫폼입니다.

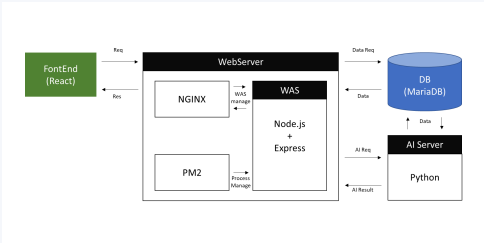
백엔드 개발자 1명과 협업해 프론트엔드를 담당했습니다.
서비스 구성도와 라벨링 화면을 중심으로 API 연동 범위와 좌표 변환 방식을 설명합니다.

구현 구조와 설계

서비스 구성과 프론트엔드 담당 범위

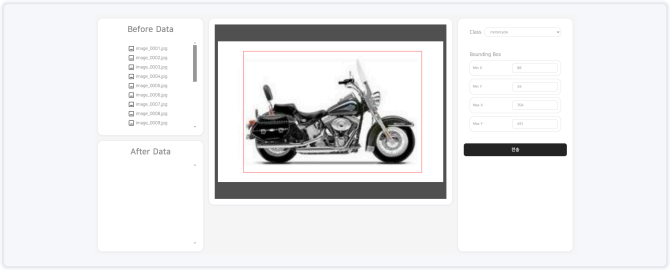
React 프론트엔드는 웹 서버 API를 통해 프로젝트·라벨·모델 데이터를 조회하고 변경합니다.
백엔드와 AI 서버가 연결된 전체 구성에서 프론트엔드 화면과 API 연동을 담당했습니다.

API 호출을 auth·project·class·datamanage·aiModel·operation 등 도메인별 서비스 모듈로 나눴습니다.
축소된 화면에서 그린 좌표는 원본 이미지 좌표로 변환해 저장하고, 다시 표시할 때 역변환했습니다.
양방향 변환은 useCoordinateTransform 훅으로 분리해 그리기·저장·재표시에 같은 규칙을 적용했습니다.

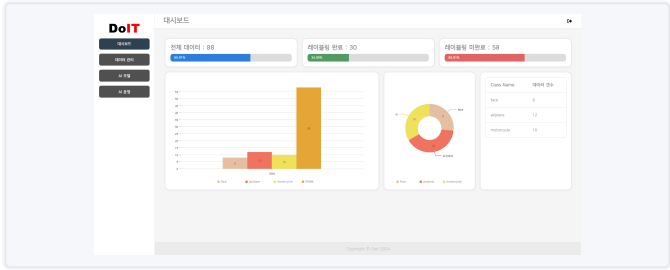


D.A.X 전체 서비스 구성 — 담당 범위는 React 프론트엔드와 API 연동

D.A.X · 프로젝트 화면



D.A.X에서 이미지 위에 바운딩 박스를 그려 라벨링하는 화면



D.A.X의 프로젝트 데이터와 학습 현황 대시보드

기술 스택

프론트엔드 TypeScript · React · styled-components · TanStack Query · Zustand

XRGIS 백오피스 XR·GIS 원격 협업 시스템 백오피스

2023.08 – 2023.12 프론트엔드·백엔드 개발 담당

XR·GIS 원격 협업 서비스의 사용자·팀·파일 관리 백오피스입니다.

React 화면과 Express API를 담당했습니다.
아래 화면은 사용자·팀 관리와 팀별 파일 접근, 대용량 파일 처리 기능을 보여줍니다.

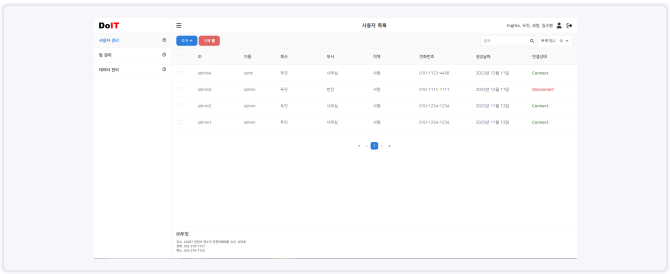
구현 구조와 설계

팀별 접근 권한과 대용량 파일 처리

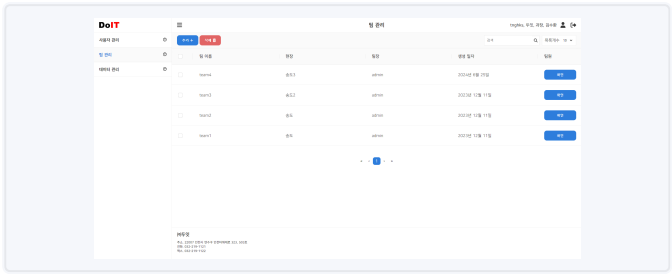
파일 관리 API에서 사용자의 팀 소속에 따라 다운로드 권한을 확인합니다.
같은 관리자 계정을 사용하는 사내 서비스와는 MariaDB 기반 세션 저장소를 통해 인증 체계를 맞췄습니다.

OBJ·영상 파일은 MongoDB GridFS로 청크를 나눠 저장하고, 선택한 파일은 archiver로 ZIP을 생성해 다운로드하도록 구현했습니다.

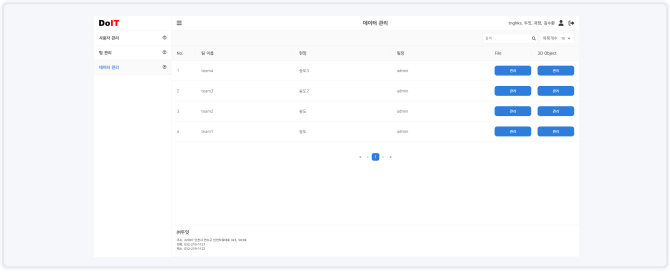
XRGIS 백오피스 · 프로젝트 화면



XRGIS 사용자 관리 화면



XRGIS 팀 관리 화면



XRGIS 팀별 데이터·파일 관리 화면: 대용량 파일 관리와 ZIP 다운로드 기능

기술 스택

프론트엔드	TypeScript · React · styled-components
백엔드	Node.js · Express · MongoDB(GridFS) · MariaDB